微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

附录B　展望Java技术的未来（2013年版）

本书第1版和2版中的“展望Java技术的未来”分别成文于2011年和2013年，近十年时间已经过去，当时畅想的Java新发展新变化全部如约而至，这部分内容已不再有“展望”的价值。笔者在更新第3版时重写了全部相关内容，并把第2版的“展望”的原文挪到附录之中。假若Java的未来依旧灿烂精彩，假若下一个十年本书还会有第4、第5版，那希望届时能在附录中回首今日，去回溯哪些预测成为现实，哪些改进中途夭折。

在2005年，Java语言诞生10周年的SunOne技术大会上，Java语言之父James Gosling做过题为《Java技术下一个十年》的演讲。笔者不具备James Gosling博士那样高屋建瓴的视角，这里仅从Java平台中几个新生的但已经开始展现出蓬勃之势的技术发展点来看一下后续一至两个JDK版本内的一些很有希望的技术重点。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

B.1 模块化

模块化是解决应用系统与技术平台越来越复杂、越来越庞大的一个重要途径。无论是开发人员还是产品最终用户，都不希望为了系统中一小块的功能而不得不下载、安装、部署及维护整套庞大的系统。站在整个软件工业化的高度来看，模块化是建立各种功能标准件的前提。最近几年OSGi技术的迅速发展、各个厂商在JCP中对模块化规范的激烈斗争[1]，都能充分说明模块化技术的迫切和重要。

在未来的Java平台中，很可能会对模块化提出语法层面的支持。早在2007年，Sun公司就提出过JSR-277：Java模块系统（Java Module System），试图建立Java平台的模块化标准，但受挫于以IBM为主导的提交的JSR-291：Java SE动态组件支持（Dynamic Component Support for Java SE，实际就是OSGi R4.1）。由于模块化规范主导权的重要性，Sun不能接受一个无法由它控制的规范，在整个Java SE 6期间都拒绝把任何模块化技术内置到JDK之中。在Java SE 7发展初期，Sun公司再次提交了一个新的规范请求文档JSR-294：Java编程语言中的改进模块性支持（Improved Modularity Support in the Java Programming Language），尽管这个JSR仍然没有通过，但是Sun已经独立于JCP专家组在OpenJDK里建立了一个名为Jigsaw（拼图）的子项目来将这个规范在Java平台中转变为具体的实现。Java的模块化之争目前还没有结束，OSGi已经发布到R5.0版本，而Jigsaw从Java 7延迟至Java 8，在2012年7月又不得不宣布推迟到Java 9中发布，从这一点看来，Sun在这场战争中处于劣势，但无论胜利者是哪一方，Java模块化已经成为一股无法阻挡的变革潮流。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

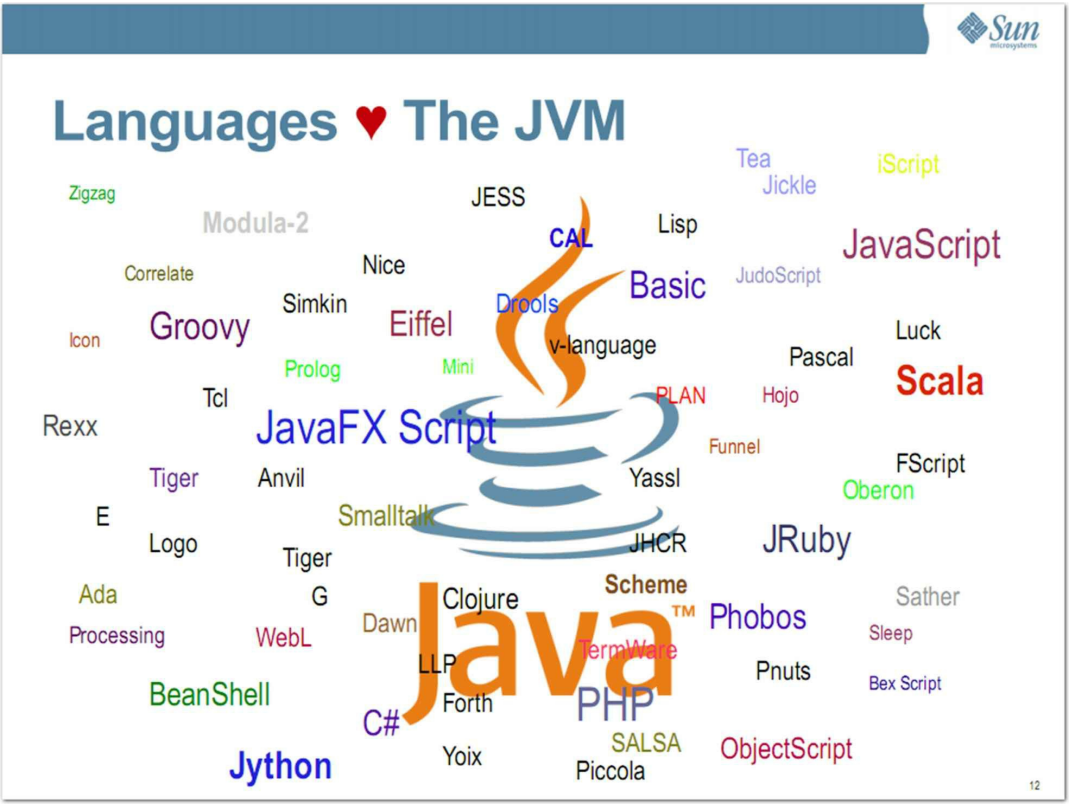
B.2 混合语言

当单一的Java开发已经无法满足当前软件复杂的需求时，越来越多基于Java虚拟机的开发语言被应用到软件项目中，Java平台上的多语言混合编程正成为主流，每种语言都可以针对自己擅长的方面更好地解决问题。试想一下：在一个项目之中，并行处理用Clojure语言编写，展示层使用JRuby/Rails，中间层则是Java，每个应用层都使用不同的编程语言来完成，而且，接口对每一层的开发者都是透明的，各种语言之间的交互不存在任何困难，就像使用自己语言的原生API一样方便[2]，因为他们最终都运行在一个虚拟机之上。

在最近的几年里，Clojure、JRuby、Groovy等新生语言的使用人数如同滚动的雪球一般增长，而运行在Java虚拟机之上的语言数量也在迅速膨胀，图B-1中列举了其中的一部分。这两点证明混合编程在我们身边已经有所应用并被广泛认可。通过特定领域的语言去解决特定领域的问题是当前软件开发应对日趋复杂的项目需求的一个方向。

除了催生大量的新语言外，许多已经有很长历史的程序语言也出现了基于Java虚拟机实现的版本，这样混合编程对许多以前使用其他语言的“老”程序员也具备相当大的吸引力，软件企业投入了大量资本的现有代码资产也能被很好地保护起来。表B-1中列举了常见语言的Java虚拟机实现版本。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程



图B-1　可以运行在JVM之上的语言[3]

表B-1　常见语言的JVM实现版本

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

语　　言	基于 JVM 实现的版本
Ada	JGNAT
AWK	Jawk
C	C to Java Virtual Machine compilers
Cobol	Veryant isCobol
ColdFusion	Adobe ColdFusion、Railo、Open BlueDragon
Common Lisp	Armed Bear Common Lisp、CLforJava、Jatha (Common LISP)
Component Pascal	Gardens Point Component Pascal
Erlang	Erjang
Forth	myForth
JavaScript	Rhino
LOGO	jLogo、XLogo
Lua	Kahlua、Luaj、Jill
Oberon-2	Canterbury Oberon-2 for JVM
Objective Caml (OCaml)	OCaml-Java
Pascal	Canterbury Pascal for JVM

（续）

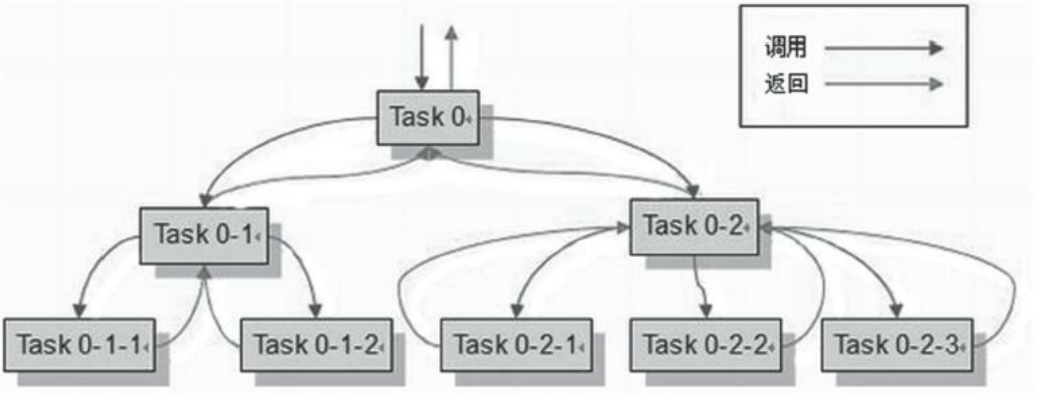
语　　言	基于 JVM 实现的版本
PHP	IBM WebSpheresMash PHP (P8)、CauchoQuercus
Python	Jython
Rexx	IBM NetRexx
Ruby	JRuby
Scheme	Bigloo、Kawa、SISC、JScheme

对这些运行于Java虚拟机之上、Java之外的语言，来自系统级的、底层的支持正在迅速增强，以JSR-292为核心的一系列项目和功能改进（如Da Vinci Machine项目、Nashorn引擎、InovkeDynamic指令、java.lang.invoke包等），推动Java虚拟机从“Java语言的虚拟机”向“多语言虚拟机”的方向发展。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

B.3　多核并行

如今，CPU硬件的发展方向已经从高频率转变为多核心，随着多核时代的来临，软件开发越来越关注并行编程的领域。早在JDK 1.5之中就已经引入java.util.concurrent包，实现了一个粗粒度的并发框架。而JDK 1.7中加入的java.util.concurrent.forkjoin包则是对这个框架的一次重要扩充。Fork/Join模式是处理并行编程的一个经典方法，如图B-2所示。虽然不能解决所有的问题，但是在它的适用范围之内，能够轻松地利用多个CPU核心提供的计算资源来协作完成一个复杂的计算任务。通过利用Fork/Join模式，我们能够更加顺畅地过渡到多核时代。



图B-2　Fork/Join模式示意图[4]

在Java 8中，将会提供Lambda支持，将会极大改善目前Java语言不适合函数式编程的现状（目前Java语言使用函数式编程并不是不可以，只是会显得很臃肿），函数式编程的一个重要优点就是这样的程序天然地适合并行运行，这样对Java语言在多核时代继续保持主流语言的地位有很大帮助。

另外并行计算中必须提及的还有OpenJDK的子项目Sumatra[5]，目前显卡的算术运算能力、并行能力已经远远超过了CPU，在图形领域以外发掘显卡的潜力是近几年计算机发展的方向之一，例如C语言的CUDA。Sumatra项目就是为Java提供使用GPU（Graphics Processing Unit）和APU（Accelerated Processing Unit）运算能力的工具，以后它将会直接提供Java语言层面的API，或者为Lambda和其他JVM语言提供底层的并行运算支持。

在JDK外围，也出现了专为实现并行计算需求的计算框架，如Apache的Hadoop Map/Reduce，这是一个简单易懂的并行框架，能够运行在由上千个商用机器组成的大型集群上，并能以一种可靠的容错方式并行处理TB级别的数据集。另外，还出现了诸如Scala、Clojure及Erlang等天生就具备并行计算能力的语言。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

B.4 进一步丰富语法

Java 5曾经对Java语法进行了一次扩充，这次扩充加入了自动装箱、泛型、动态注解、枚举、可变长参数、遍历循环等语法，使得Java语言的精确性和易用性有了很大的进步。在Java 7（由于进度压力，许多改进已被推迟至Java 8）中，对Java语法进行了另一次大规模的扩充。Sun（Oracle）专门为改进Java语法在OpenJDK中建立了Coin子项目[6]来统一处理Java语法的细节修改，如对二进制数的原生支持、在switch语句中支持字符串、“<>”操作符、异常处理的改进、简化变长参数方法调用、面向资源的try-catch-finally语句等都是在Coin项目之中提交的内容。

除了Coin项目之外，JSR-335（Lambda Expressions for the JavaTM Programming Language）中定义的Lambda表达式[7]，也将对Java的语法和语言习惯产生很大的影响，面向函数方式的编程可能会成为主流。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

B.5　64位虚拟机

几年之前，主流的CPU就开始支持64位架构。Java虚拟机也在很早之前就推出了支持64位系统的版本。但Java程序运行在64位虚拟机上需要付出比较大的额外代价：首先是内存问题，由于指针膨胀和各种数据类型对齐补白的原因，运行于64位系统上的Java应用需要消耗更多的内存，通常要比32位系统额外增加10%～30%的内存消耗；其次是多个机构的测试结果显示，64位虚拟机的运行速度在各个测试项上几乎都全面落后于32位虚拟机，两者大约有15%的性能差距。

但是在Java EE方面，企业级应用经常需要使用超过4GB的内存，对于64位虚拟机的需求是非常迫切的，但由于上述的原因，许多企业应用都仍然选择使用虚拟集群等方式继续在32位虚拟机中进行部署。Sun也注意到了这些问题，并做出了一些改善，在JDK 1.6 Update 14之后，提供了普通对象指针压缩功能（-XX：+UseCompressedOops，这个参数不建议显式设置，建议维持默认由虚拟机的Ergonomics机制自动开启），在执行代码时，动态植入压缩指令以节省内存消耗。但是开启压缩指针会增加执行代码数量，因为所有在Java堆里的、指向Java堆内对象的指针都会被压缩，这些指针的访问就需要更多的代码才可以实现，而且并不仅只是读写字段才受影响，在实例方法调用、子类型检查等操作中也受影响，因为对象实例指向对象类型的引用也被压缩了。随着硬件的进一步发展，计算机终究会完全过渡到64位的时代，这是一件毫无疑问的事情，主流的虚拟机应用也终究会从32位发展至64位，而虚拟机对64位的支持也将会进一步完善。

[1] 如果读者对Java模块化之争感兴趣，可以阅读作者的另外一本书《深入理解OSGi》的第1章。
[2] 在同一个虚拟机上跑的其他语言与Java之间的交互一般都比较容易，但非Java语言之间的交互一般都比较烦琐。dynalang项目（http://dynalang.sourceforge.net/）就是为了解决这个问题而出现的。
[3] 图片来源：http://wikis.sun.com/download/attachments/16418319/OOW-2009+Towards+A+Universal+VM.pdf。
[4] 图片来源：http://www.ibm.com/developerworks/cn/java/j-lo-forkjoin/。
[5] Sumatra项目：http://openjdk.java.net/projects/sumatra/。
[6] Coin项目主页：https://openjdk.java.net/projects/coin。
[7] Lambda项目主页：http://openjdk.java.net/projects/lambda/。